



Unidad Profesional Interdisciplinaria de Ingeniería y ciencias
sociales y administrativas

UPIICSA



Estructura de datos

Programas para entregar

Secuencia:

2NM23

Docente:

Entzana Garduño Yventz

Equipo:

Galindo Cantillo Franco

Díaz Gasca Sofia

Martínez Santiago Daniel

Pedro Zenón Monserrat

Índice

13. Nuevo tipo de dato con puntero al arreglo pariendo del ejercicio anterior, acceder de manera indirecta a la lista de las personas.
16. Ordenamiento burbuja: ordenar conjunto de caracteres, enteros y nuevo tipo de dato (PE y POO)
17. Ordenamiento merge sort: ordenar conjunto de caracteres, enteros y nuevo tipo de dato (PE y POO)
18. Ordenamiento quick sort: ordenar conjunto de caracteres, enteros y nuevo tipo de dato (PE y POO)
19. Ordenamiento de manera indirecta burbuja: ordenar conjunto de caracteres, enteros y nuevo tipo de dato (PE y POO)
20. Ordenamiento de manera indirecta merge sort: ordenar conjunto de caracteres, enteros y nuevo tipo de dato (PE y POO)
21. Ordenamiento de manera indirecta quick sort: ordenar conjunto de caracteres, enteros y nuevo tipo de dato (PE y POO)
1. Punteros para el manejo indirecto con clases y estructuras (como el ejercicio 12, pero dentro de la clase o estructura considerando un puntero o referencia al mismo tipo de dato)
3. PILA: Manejo de una pila de manera estática por medio de un arreglo
4. COLA: Manejo de una cola de manera estática por medio de un arreglo
5. LISTA: Manejo de una lista de manera estática por medio de un arreglo
6. PILA: Manejo de una pila de manera dinámica por medio de punteros al mismo tipo de dato de la pila, considerando la referencia
7. COLA: Manejo de una cola de manera dinámica por medio de punteros al mismo tipo de dato, considerando la referencia
- 7 bis. LISTA: Manejo de una lista de manera dinámica por medio de punteros al mismo tipo de dato, considerando la referencia
8. PILA: Manejo de una pila de manera dinámica por medio de las librerías o clases proporcionadas por el IDE
9. COLA: Manejo de una cola de manera dinámica por medio de las librerías o clases proporcionadas por el IDE
10. LISTA: Manejo de una lista de manera dinámica por medio de las librerías o clases proporcionadas por el IDE

8. PILA: Manejo de una pila de manera dinámica por medio de las librerías o clases proporcionadas por el IDE

Introducción a la práctica

En esta práctica se desarrolla un programa en C++ que permite trabajar con una estructura de datos tipo pila utilizando la librería estándar <stack>. Las pilas son estructuras fundamentales en la programación, caracterizadas por operar bajo el principio LIFO (Last In, First Out), donde el último elemento en entrar es el primero en salir. A través de un menú interactivo en consola, el usuario puede realizar operaciones básicas como insertar, eliminar y consultar elementos, lo que facilita la comprensión del comportamiento de esta estructura.

Objetivo

El objetivo de esta práctica es implementar y manipular una pila dinámica en C++ utilizando las funciones proporcionadas por la librería <stack>. Se busca que el usuario comprenda el funcionamiento de las operaciones principales de una pila: inserción (push), eliminación (pop), consulta del elemento en la cima (top) y verificación de si la pila está vacía (empty), además de reforzar el uso de estructuras de control como ciclos y condicionales.

Desarrollo

El programa inicia con la inclusión de las librerías necesarias:

- <iostream> para entrada y salida de datos.
- <stack> para utilizar la estructura de pila.

Dentro de la función principal main:

- Se declara una pila de enteros llamada pila.
- Se definen dos variables:
 - opcion: almacena la opción seleccionada por el usuario.
 - dato: almacena el valor a insertar en la pila.

El programa utiliza un ciclo do-while que permite ejecutar el menú al menos una vez y repetirlo hasta que el usuario decida salir.

El menú presenta las siguientes opciones:

- Opción 1: Insertar (push)
 - Solicita un dato al usuario.
 - Inserta el valor en la pila usando push.
- Opción 2: Eliminar (pop)
 - Verifica si la pila está vacía con empty.
 - Si no está vacía:
 - Muestra el elemento en la cima con top.
 - Elimina dicho elemento con pop.
 - Si está vacía, muestra un mensaje de advertencia.
- Opción 3: Mostrar cima (top)
 - Verifica si la pila está vacía.
 - Si contiene elementos, muestra el valor en la cima con top.
 - Si está vacía, indica que no hay elementos.
- Opción 4: Verificar si está vacía
 - Usa la función empty:
 - Si está vacía, lo indica al usuario.
 - Si no, informa que contiene elementos.
- Opción 5: Salir
 - Finaliza el ciclo y termina el programa.

El programa continúa ejecutándose hasta que el usuario selecciona la opción 5.

```

1  #include <iostream>
2  #include <stack> //librería de pila
3  using namespace std;
4
5  int main() {
6      stack<int> pila; //pila dinámica
7
8      int opcion, dato;
9
10     do {
11         cout << "\n--- MENU PILA ---" << endl;
12         cout << "1. Insertar (push)" << endl;
13         cout << "2. Eliminar (pop)" << endl;
14         cout << "3. Mostrar cima (top)" << endl;
15         cout << "4. Ver si esta vacia" << endl;
16         cout << "5. Salir" << endl;
17         cout << "Opcion: ";
18         cin >> opcion;
19
20         switch(opcion) {
21             case 1:
22                 cout << "Ingresa dato: ";
23                 cin >> dato;
24                 pila.push(dato);
25                 break;
26
27             case 2:
28                 if (!pila.empty()) {
29                     cout << "Eliminado: " << pila.top() << endl;
30                     pila.pop();
31                 } else {
32                     cout << "La pila esta vacia" << endl;
33                 }
34                 break;
35
36             case 3:
37                 if (!pila.empty()) {
38                     cout << "Cima: " << pila.top() << endl;
39                 } else {
40                     cout << "La pila esta vacia" << endl;
41                 }
42                 break;
43
44             case 4:
45                 if (pila.empty()) {
46                     cout << "La pila esta vacia" << endl;
47                 } else {
48                     cout << "La pila tiene elementos" << endl;
49                 }
50                 break;
51         }
52     } while(opcion != 5);
53
54     return 0;
55 }
56
57

```

C:\Users\franc\OneDrive\Doc

```

--- MENU PILA ---
1. Insertar (push)
2. Eliminar (pop)
3. Mostrar cima (top)
4. Ver si esta vacia
5. Salir
Opcion: 4
La pila esta vacia

```

```

--- MENU PILA ---
1. Insertar (push)
2. Eliminar (pop)
3. Mostrar cima (top)
4. Ver si esta vacia
5. Salir
Opcion: 1
Ingresa dato: 455445

```

```

--- MENU PILA ---
1. Insertar (push)
2. Eliminar (pop)
3. Mostrar cima (top)
4. Ver si esta vacia
5. Salir
Opcion:

```

Conclusión

La práctica permite entender el comportamiento de una pila y sus operaciones fundamentales de manera clara y estructurada. El uso de listas en la explicación facilita la identificación de cada funcionalidad del programa. Además, se refuerza el uso de estructuras de control y validaciones, elementos clave en el desarrollo de programas robustos. La librería <stack> simplifica la implementación, permitiendo enfocarse en la lógica de uso de la estructura en lugar de su construcción interna.

9.- COLA Manejo de una cola de manera dinámica por medio de las librerías o clases proporcionadas por el IDE

Introducción a la práctica

En esta práctica se desarrolla un programa en C++ que implementa una estructura de datos tipo cola utilizando la librería estándar <queue>. A diferencia de la pila, la cola trabaja bajo el principio FIFO (First In, First Out), donde el primer elemento en entrar es el primero en salir. El programa incluye un menú interactivo que permite al usuario realizar distintas operaciones sobre la cola, facilitando la comprensión de su funcionamiento.

Objetivo

El objetivo es comprender el uso de una cola dinámica en C++, así como sus operaciones principales. Se busca que el usuario aprenda a insertar y eliminar elementos, consultar el primero y el último valor, y verificar si la estructura está vacía, además de reforzar el uso de estructuras de control en un programa interactivo.

Desarrollo

El programa comienza con la inclusión de las librerías <iostream> y <queue>, necesarias para la entrada/salida de datos y la implementación de la cola. Se utiliza std para simplificar la sintaxis.

Dentro de main:

- Se declara una cola de enteros llamada cola.
- Se definen las variables opcion y dato para controlar el menú y almacenar valores.

El programa funciona mediante un ciclo do-while que se repite hasta que el usuario selecciona salir. El menú permite realizar las siguientes acciones:

- Insertar (opción 1):
Se solicita un valor al usuario y se agrega al final de la cola con push.
- Eliminar (opción 2):
Se verifica si la cola está vacía.
 - Si no lo está, se muestra el elemento del frente con front y luego se elimina con pop.
 - Si está vacía, se muestra un mensaje indicando que no hay elementos.
- Mostrar frente (opción 3):
Se consulta el primer elemento de la cola usando front, siempre validando que no esté vacía.
- Mostrar último (opción 4):
Se muestra el elemento más reciente insertado mediante la función back.
- Verificar si está vacía (opción 5):
Se utiliza empty para determinar si la cola contiene elementos o no.

- Salir (opción 6): Termina la ejecución del programa.

The image shows a C++ program in a code editor and its execution output in a terminal window. The code implements a queue using the STL `queue` container. It features a menu-driven interface with options to push, pop, view front, view back, check if empty, and exit. The output shows the program running successfully, demonstrating the queue's behavior.

```
main.cpp ×
1 #include <iostream>
2 #include <queue> // Librería de cola
3 using namespace std;
4
5 int main() {
6     queue<int> cola; //cola dinámica
7
8     int opcion, dato;
9
10    do {
11        cout << "\n--- MENU COLA ---" << endl;
12        cout << "1. Insertar (push)" << endl;
13        cout << "2. Eliminar (pop)" << endl;
14        cout << "3. Mostrar frente (front)" << endl;
15        cout << "4. Mostrar ultimo (back)" << endl;
16        cout << "5. Ver si esta vacia" << endl;
17        cout << "6. Salir" << endl;
18        cout << "Opcion: ";
19        cin >> opcion;
20
21        switch(opcion) {
22            case 1:
23                cout << "Ingresa dato: ";
24                cin >> dato;
25                cola.push(dato);
26                break;
27
28            case 2:
29                if (!cola.empty()) {
30                    cout << "Eliminado: " << cola.front() << endl;
31                    cola.pop();
32                } else {
33                    cout << "La cola esta vacia" << endl;
34                }
35                break;
36
37            case 3:
38                if (!cola.empty()) {
39                    cout << "Frente: " << cola.front() << endl;
40                } else {
41                    cout << "La cola esta vacia" << endl;
42                }
43                break;
44
45            case 4:
46                if (!cola.empty()) {
47                    cout << "Ultimo: " << cola.back() << endl;
48                } else {
49                    cout << "La cola esta vacia" << endl;
50                }
51                break;
52
53            case 5:
54                if (cola.empty()) {
55                    cout << "La cola esta vacia" << endl;
56                } else {
57                    cout << "La cola tiene elementos" << endl;
58                }
59                break;
60        }
61    } while(opcion != 6);
62
63 }
```

```
C:\Users\franc\OneDrive\Doc × + v
La cola esta vacia

--- MENU COLA ---
1. Insertar (push)
2. Eliminar (pop)
3. Mostrar frente (front)
4. Mostrar ultimo (back)
5. Ver si esta vacia
6. Salir
Opcion: 1
Ingresa dato: 323232

--- MENU COLA ---
1. Insertar (push)
2. Eliminar (pop)
3. Mostrar frente (front)
4. Mostrar ultimo (back)
5. Ver si esta vacia
6. Salir
Opcion: 4
Ultimo: 323232

--- MENU COLA ---
1. Insertar (push)
2. Eliminar (pop)
3. Mostrar frente (front)
4. Mostrar ultimo (back)
5. Ver si esta vacia
6. Salir
Opcion: |
```

Conclusión

Esta práctica permite comprender de manera clara cómo funciona una cola y sus operaciones principales. El uso de listas ayuda a identificar fácilmente cada acción disponible en el programa. Además, se refuerzan conceptos importantes como el control de flujo, validación de condiciones y manipulación de estructuras dinámicas, lo cual es fundamental en la programación.

10.- Manejo de una lista de manera dinámica por medio de las librerías o clases proporcionadas por el IDE

Introducción a la práctica

En esta práctica se desarrolla un programa en C++ que utiliza la estructura de datos tipo lista mediante la librería estándar `<list>`. A diferencia de estructuras como la pila o la cola, la lista permite insertar y eliminar elementos tanto al inicio como al final de manera eficiente. El programa implementa un menú interactivo en consola que permite al usuario manipular la lista y observar su comportamiento dinámico, facilitando la comprensión de este tipo de estructura.

Objetivo

El objetivo de esta práctica es comprender el funcionamiento de una lista en C++ y aprender a manipularla utilizando las funciones que ofrece la librería `<list>`. Además, se busca reforzar el uso de estructuras de control, validación de entradas y recorridos de estructuras dinámicas mediante iteradores.

Desarrollo

El programa comienza incluyendo las librerías `<iostream>` y `<list>`, necesarias para la interacción con el usuario y el manejo de la lista. Se utiliza el espacio de nombres `std` para simplificar la sintaxis.

Dentro de la función `main`, se declara una lista de enteros llamada `lista`, junto con las variables `opcion` y `dato`, que permiten controlar el flujo del programa y almacenar valores ingresados por el usuario.

El programa se ejecuta mediante un ciclo `do-while`, el cual mantiene activo el menú hasta que el usuario decide salir. Antes de procesar la opción elegida, se valida la entrada con `cin.fail()` para evitar errores en caso de que el usuario introduzca un valor no numérico; en ese caso, se limpia el estado de entrada y se asigna una opción inválida.

Las operaciones disponibles en el menú son las siguientes:

- Insertar al inicio (opción 1):
Se solicita un valor y se agrega al inicio de la lista utilizando `push_front`.
- Insertar al final (opción 2):
Se solicita un valor y se agrega al final de la lista mediante `push_back`.
- Eliminar inicio (opción 3):
Se verifica si la lista contiene elementos.
 - Si no está vacía, se elimina el primer elemento con `pop_front`.
 - Si está vacía, se muestra un mensaje indicando la situación.
- Eliminar final (opción 4):
Se valida que la lista no esté vacía.
 - Si tiene elementos, se elimina el último con `pop_back`.

- En caso contrario, se notifica que la lista está vacía.
- Mostrar lista (opción 5):
Se recorre la lista utilizando un iterador desde begin() hasta end(), mostrando cada elemento almacenado. Si la lista está vacía, se indica al usuario.
- Verificar si está vacía (opción 6):
Se utiliza la función empty para determinar si la lista contiene elementos o no, mostrando el mensaje correspondiente.

El ciclo continúa ejecutándose hasta que el usuario selecciona la opción 7, que finaliza el programa

The screenshot shows a C++ IDE with a file named `main.cpp` and a console window. The code implements a linked list using `list<int>` and a menu-driven interface.

```

1  #include <iostream>
2  #include <list>
3  using namespace std;
4
5  int main() {
6      list<int> lista;
7
8      int opcion, dato;
9
10     do {
11         cout << "\n--- MENU LISTA ---" << endl;
12         cout << "1. Insertar al inicio" << endl;
13         cout << "2. Insertar al final" << endl;
14         cout << "3. Eliminar inicio" << endl;
15         cout << "4. Eliminar final" << endl;
16         cout << "5. Mostrar lista" << endl;
17         cout << "6. Ver si esta vacia" << endl;
18         cout << "7. Salir" << endl;
19         cout << "Opcion: ";
20
21         cin >> opcion;
22
23         if (cin.fail()) {
24             cin.clear();
25             cin.ignore(1000, '\n');
26             opcion = 0;
27         }
28
29         switch(opcion) {
30             case 1:
31                 cout << "Dato: ";
32                 cin >> dato;
33                 lista.push_front(dato);
34                 break;
35
36             case 2:
37                 cout << "Dato: ";
38                 cin >> dato;
39                 lista.push_back(dato);
40                 break;
41
42             case 3:
43                 if (!lista.empty()) {
44                     lista.pop_front();
45                 } else {
46                     cout << "Lista vacia" << endl;
47                 }
48                 break;
49
50             case 4:
51                 if (!lista.empty()) {
52                     lista.pop_back();
53                 } else {
54                     cout << "Lista vacia" << endl;
55                 }
56                 break;
57
58             case 5:
59                 if (!lista.empty()) {
60                     cout << "Lista: ";
61                     for (list<int>::iterator it = lista.begin(); it != lista.end(); it++) {
62                         cout << *it << " ";
63                     }
64                     cout << endl;
65                 }
66
67             default:
68                 break;
69         }
70     } while (opcion != 7);
71 }

```

The console output shows the menu being displayed three times, corresponding to the user's selections of options 6, 1, and 2. The output for option 6 is "Lista vacia". The output for option 1 is "Dato: 232323". The output for option 2 is "Dato: 454544".

Conclusión

Esta práctica permite comprender el uso de listas dinámicas en C++ y las ventajas que ofrecen para la inserción y eliminación de elementos en diferentes posiciones. También se refuerza el uso de iteradores para recorrer estructuras y la validación de datos para evitar errores en la ejecución. El programa demuestra cómo manipular una lista de manera flexible y controlada mediante un entorno interactivo.



Instituto Politécnico Nacional
Unidad Profesional
Interdisciplinaria de Ingeniería y
Ciencias Sociales y
Administrativas



Prácticas

Segundo Parcial

Integrantes
Martínez Santiago Daniel

No. Lista
21

Profesor: Yventz Entzana Garduño

Secuencia: 2NM23

Repositorio de GitHub:

<https://github.com/DaniRex95/Estructura-de-Datos-Ejercicios.git>

Ejercicio 3: Pila

Descripción del programa:

El programa implementa una estructura de datos tipo pila utilizando un arreglo estático en lenguaje C++.

Una pila sigue el principio LIFO (Last In, First Out), lo que significa que el último elemento en entrar es el primero en salir.

El sistema permite al usuario interactuar mediante un menú en consola para realizar operaciones básicas como:

- Insertar elementos (push)
- Eliminar elementos (pop)
- Mostrar el contenido
- Ordenar los datos mediante tres algoritmos:
 - Burbuja
 - QuickSort
 - MergeSort

El programa está dividido en múltiples archivos (.h y .c) para cumplir con una estructura modular, separando la interfaz de la implementación.

Descripción de las clases:

Clase Pila.h

El archivo Pila.h es un encabezado que define la estructura y las operaciones de una pila utilizando un arreglo de tamaño fijo. Su propósito es declarar los elementos necesarios para que otros archivos del programa puedan usar esta estructura sin conocer su implementación interna.

Primero, se utilizan directivas de preprocesador (`#ifndef`, `#define`, `#endif`) para evitar que el archivo sea incluido más de una vez durante la compilación, lo que previene errores.

Se define la constante `MAX` con valor 50, que establece la capacidad máxima de la pila.

Posteriormente, se declara la estructura `Pila`, la cual contiene:

- Un arreglo `datos` donde se almacenan los elementos.
- Una variable `tope` que indica la posición del último elemento insertado.

La pila sigue el principio **LIFO (Last In, First Out)**, es decir, el último elemento en entrar es el primero en salir.

Finalmente, se declaran las funciones que permiten manipular la pila, como inicializarla, insertar (push), eliminar (pop), verificar si está vacía o llena, mostrar sus elementos y obtener su tamaño. Estas funciones se implementan en otro archivo, pero aquí se establecen sus prototipos para su uso.

```
1  #ifndef PILA_H
2  #define PILA_H
3
4  #define MAX 50
5
6  typedef struct {
7      int datos[MAX];
8      int tope;
9  } Pila;
10
11 void inicializar(Pila *p);
12 void push(Pila *p, int valor);
13 int pop(Pila *p);
14 int estaVacia(Pila *p);
15 int estaLlena(Pila *p);
16 void mostrar(Pila *p);
17 int tamano(Pila *p);
18
19 #endif
```

Clase Pila.c

El archivo Pila.c contiene la implementación de las funciones declaradas en Pila.h. Aquí es donde se define el comportamiento real de la pila.

La función inicializar establece el valor de tope en -1, lo que indica que la pila está vacía.

La función push agrega un elemento a la pila. Primero verifica que aún haya espacio ($\text{tope} < \text{MAX} - 1$), luego incrementa el índice tope y guarda el valor en el arreglo. Si la pila está llena, muestra un mensaje de error.

La función pop elimina y retorna el elemento en la cima de la pila. Antes de hacerlo, verifica que la pila no esté vacía. Si lo está, muestra un mensaje y devuelve -1.

La función estaVacia debería verificar si la pila no tiene elementos. Sin embargo, en este código hay un error: se está comparando $\text{tope} == \text{MAX} - 1$, lo cual en realidad corresponde a una pila llena, no vacía. Lo correcto sería comparar con -1.

La función mostrar imprime los elementos de la pila desde el tope hacia abajo, respetando el comportamiento LIFO. Antes de recorrerla, valida si está vacía.

Finalmente, tamaño devuelve la cantidad de elementos almacenados, calculada como tope + 1.

```
1 #include <stdio.h>
2 #include "pila.h"
3
4 void inicializar(Pila *p) {
5     p->tope = -1;
6 }
7
8 void push(Pila *p, int valor) {
9     if (p->tope < MAX - 1) {
10         p->datos[++p->tope] = valor;
11         printf("Elemento agregado \n");
12     } else {
13         printf("Pila llena \n");
14     }
15 }
16
17 int pop(Pila *p) {
18     if (!estaVacia(p)) {
19         return p->datos[p->tope--];
20     }
21     printf("Pila vacia \n");
22     return -1;
23 }
24
25 int estaVacia(Pila *p) {
26     return p->tope == MAX - 1;
27 }
28
29 void mostrar(Pila *p) {
30     if (estaVacia(p)) {
31         printf("Pila vacia \n");
32         return;
33     }
34
35     for (int i = p->tope; i >= 0; i--) {
36         printf("%d \n", p->datos[i]);
37     }
38 }
39
40 int tamaño(Pila *p) {
41     return p->tope + 1;
42 }
```

Clase Ordenamientos.h

El archivo Ordenamientos.h es un encabezado que declara los prototipos de tres algoritmos de ordenamiento: Burbuja, Quick Sort y Merge Sort. Su función es permitir que estos métodos puedan ser utilizados desde otros archivos sin necesidad de conocer su implementación interna.

Cada función recibe un arreglo de enteros como parámetro, junto con los índices necesarios para definir el rango que se desea ordenar. En el caso de burbuja, se pasa el tamaño del arreglo, mientras que en quickSort y mergeSort se utilizan índices (inicio y fin) para trabajar de forma recursiva.

Este archivo no contiene lógica de ordenamiento, únicamente define la interfaz de uso de los algoritmos.

```
1 #ifndef ORDENAMIENTOS_H
2 #define ORDENAMIENTOS_H
3
4 void burbuja(int arr[], int n);
5 void quickSort(int arr[], int low, int high);
6 void mergeSort(int arr[], int left, int right);
7
8 #endif
```

Clase Ordenamientos.c

El archivo Ordenamientos.c contiene la implementación de tres algoritmos de ordenamiento: Burbuja, Quick Sort y Merge Sort.

El método burbuja ordena el arreglo comparando elementos adyacentes e intercambiándolos si están en el orden incorrecto. Es un algoritmo sencillo, pero poco eficiente para grandes volúmenes de datos.

El algoritmo Quick Sort utiliza un enfoque recursivo basado en “divide y vencerás”. Selecciona un pivote (último elemento), reorganiza el arreglo colocando los valores menores a la izquierda y mayores a la derecha mediante la función particion, y luego aplica el mismo proceso a los subarreglos generados.

El algoritmo Merge Sort también sigue el enfoque de “divide y vencerás”. Divide el arreglo en partes más pequeñas hasta llegar a elementos individuales, y luego los combina ordenadamente mediante la función merge, utilizando un arreglo auxiliar.

Un aspecto importante es que en la función merge se utiliza un arreglo temporal de tamaño fijo (temp[50]), lo cual limita el tamaño máximo de los datos a ordenar. Para un uso más general, este tamaño debería adaptarse dinámicamente.

```

1 #include "ordenamientos.h"
2
3 //Burbuja
4 void burbuja(int arr[], int n) {
5     for (int i = 0; i < n - 1; i++) {
6         for (int j = 0; j < n - i - 1; j++) {
7             if (arr[j] > arr[j + 1]) {
8                 int temp = arr[j];
9                 arr[j] = arr[j + 1];
10                arr[j + 1] = temp;
11            }
12        }
13    }
14 }
15
16 // Quick Sort
17 int particion(int arr[], int low, int high) {
18     int pivote = arr[high];
19     int i = low - 1;
20
21     for (int j = low; j < high; j++) {
22         if (arr[j] < pivote) {
23             i++;
24             int temp = arr[i];
25             arr[i] = arr[j];
26             arr[j] = temp;
27         }
28     }
29
30     int temp = arr[i + 1];
31     arr[i + 1] = arr[high];
32     arr[high] = temp;
33
34     return i + 1;
35 }
36
37 void quickSort(int arr[], int low, int high) {
38     if (low < high) {
39         int pi = particion(arr, low, high);
40         quickSort(arr, low, pi - 1);
41         quickSort(arr, pi + 1, high);
42     }
43 }

```

```

44
45 // Merge Sort
46 void merge(int arr[], int left, int mid, int right) {
47     int i = left, j = mid + 1, k = 0;
48     int temp[50];
49
50     while (i <= mid && j <= right) {
51         if (arr[i] < arr[j]) temp[k++] = arr[i++];
52         else temp[k++] = arr[j++];
53     }
54
55     while (i <= mid) temp[k++] = arr[i++];
56     while (j <= right) temp[k++] = arr[j++];
57
58     for (i = left, k = 0; i <= right; i++, k++) {
59         arr[i] = temp[k];
60     }
61 }
62
63 void mergeSort(int arr[], int left, int right) {
64     if (left < right) {
65         int mid = (left + right) / 2;
66         mergeSort(arr, left, mid);
67         mergeSort(arr, mid + 1, right);
68         merge(arr, left, mid, right);
69     }
70 }

```

Clase main.cpp

El archivo main.cpp contiene el programa principal que permite interactuar con la estructura de datos pila y aplicar distintos algoritmos de ordenamiento sobre sus elementos.

Primero, se incluyen las bibliotecas necesarias: stdio.h para entrada y salida de datos, y los archivos pila.h y ordenamientos.h para utilizar las funciones previamente definidas.

En el inicio del programa se declara una variable de tipo Pila y se inicializa mediante la función inicializar, dejando la pila vacía.

El programa funciona mediante un menú interactivo controlado por un ciclo do-while, el cual se ejecuta hasta que el usuario decide salir. Las opciones permiten:

- Insertar elementos en la pila (push)
- Eliminar el elemento superior (pop)
- Mostrar el contenido de la pila
- Ordenar los elementos utilizando tres algoritmos distintos: Burbuja, Quick Sort y Merge Sort

Para los métodos de ordenamiento, se trabaja directamente con el arreglo interno de la pila (p.datos) y se utiliza la función tamaño para determinar cuántos elementos deben ordenarse.

Un detalle que considerar es que, si la pila está vacía, algunos métodos podrían recibir valores no válidos (como índices negativos en QuickSort o MergeSort), por lo que en una versión más robusta se debería validar que la pila tenga elementos antes de ordenar.

```
1 #include <stdio.h>
2 #include "pila.h"
3 #include "ordenamientos.h"
4
5 int main() {
6     Pila p;
7     inicializar(&p);
8
9     int opcion, num;
10
11     do {
12         printf("\n----- MENU ----- \n");
13         printf("1. Push \n");
14         printf("2. Pop \n");
15         printf("3. Mostrar \n");
16         printf("4. Ordenar Burbuja \n");
17         printf("5. Ordenar QuickSort \n");
18         printf("6. Ordenar MergeSort \n");
19         printf("7. Salir \n");
20         printf("Opcion: ");
21         scanf("%d", &opcion);
22
23         switch (opcion) {
24             case 1:
25                 printf("Numero: ");
26                 scanf("%d", &num);
27                 push(&p, num);
28                 break;
29
30             case 2:
31                 printf("Eliminado: %d\n", pop(&p));
32                 break;
33
34             case 3:
35                 mostrar(&p);
36                 break;
37
38             case 4:
39                 burbuja(p.datos, tamano(&p));
40                 printf("Ordenado con Burbuja\n");
41                 mostrar(&p);
42                 break;
43 }
```

```

44     case 5:
45         quickSort(p.datos, 0, tamano(&p) - 1);
46         printf("Ordenado con QuickSort\n");
47         mostrar(&p);
48         break;
49
50     case 6:
51         mergeSort(p.datos, 0, tamano(&p) - 1);
52         printf("Ordenado con MergeSort\n");
53         mostrar(&p);
54         break;
55
56     case 7:
57         printf("Saliendo...\n");
58         break;
59
60     default:
61         printf("Opción invalida\n");
62     }
63 } while (opcion != 7);
64
65 return 0;
66 }

```

Ejecución del programa:

Se insertaron varios elementos utilizando la opción 1 (Push), en el siguiente orden:

1, 16, 54, 32 y 29.

```

----- MENU -----
1. Push
2. Pop
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 1
Numero: 1
Elemento agregado

----- MENU -----
1. Push
2. Pop
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 1
Numero: 16
Elemento agregado

```

```

----- MENU -----
1. Push
2. Pop
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 1
Numero: 54
Elemento agregado

----- MENU -----
1. Push
2. Pop
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 1
Numero: 32
Elemento agregado

```

```

----- MENU -----
1. Push
2. Pop
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 1
Numero: 29
Elemento agregado

```

Al seleccionar la opción 3 (Mostrar), los datos se imprimen como:

29, 32, 54, 16, 1

Esto demuestra que la pila respeta el comportamiento LIFO (Last In, First Out), ya que el último elemento insertado (29) es el primero en mostrarse.

```
----- MENU -----
1. Push
2. Pop
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 3
29
32
54
16
1
```

Después, se prueba el ordenamiento con la opción 4 (Burbuja). El resultado mostrado es:

54, 32, 29, 16, 1

Aunque el algoritmo ordena el arreglo internamente de menor a mayor, la impresión se realiza desde el tope hacia abajo, por lo que visualmente aparece en orden descendente.

```
----- MENU -----
1. Push
2. Pop
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 4
Ordenado con Burbuja
54
32
29
16
1
```

Las opciones 5 (QuickSort) y 6 (MergeSort) producen el mismo resultado:

54, 32, 29, 16, 1

Esto confirma que los tres algoritmos están funcionando correctamente sobre los mismos datos.

```
----- MENU -----
1. Push
2. Pop
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 5
Ordenado con QuickSort
54
32
29
16
1
```

```
----- MENU -----
1. Push
2. Pop
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 6
Ordenado con MergeSort
54
32
29
16
1
```

Ejercicio 4: Cola

Descripción del programa:

El programa implementa una estructura de datos tipo cola utilizando un arreglo estático en lenguaje C++.

Una cola sigue el principio FIFO (First In, First Out), lo que significa que el primer elemento en entrar es el primero en salir.

El sistema permite al usuario interactuar mediante un menú en consola para realizar operaciones como:

- Insertar elementos (enqueue)
- Eliminar elementos (dequeue)
- Mostrar el contenido
- Ordenar los datos mediante:
 - Burbuja
 - QuickSort
 - MergeSort

El programa está organizado en múltiples archivos (.h y .c), separando la interfaz de la implementación.

Descripción de las clases:

Clase Cola.h

El archivo Cola.h es un encabezado que define la estructura y las operaciones de una cola (queue) utilizando un arreglo de tamaño fijo.

Se utilizan directivas de preprocesador (`#ifndef`, `#define`, `#endif`) para evitar múltiples inclusiones del archivo durante la compilación.

Se define la constante `MAX` con valor 50, que indica el número máximo de elementos que puede almacenar la cola.

La estructura Cola contiene:

- Un arreglo de datos donde se almacenan los elementos.
- Un índice frente, que apunta al primer elemento de la cola.
- Un índice final, que indica la posición del último elemento insertado.

A diferencia de la pila, la cola sigue el principio FIFO (First In, First Out), donde el primer elemento en entrar es el primero en salir.

Finalmente, se declaran las funciones necesarias para manipular la cola, como inicializarla, insertar (enqueue), eliminar (dequeue), verificar su estado y mostrar su contenido.

```
1  #ifndef COLA_H
2  #define COLA_H
3
4  #define MAX 50
5
6  typedef struct {
7      int datos[MAX];
8      int frente;
9      int final;
10 } Cola;
11
12 void inicializar(Cola *c);
13 void enqueue(Cola *c, int valor);
14 int dequeue(Cola *c);
15 int estaVacia(Cola *c);
16 int estaLlena(Cola *c);
17 void mostrar(Cola *c);
18 int tamano(Cola *c);
19
20 #endif
```

Clase Cola.cpp

El archivo Cola.cpp contiene la implementación de las funciones definidas en Cola.h, permitiendo el manejo de una cola basada en arreglo.

La función inicializar establece frente = 0 y final = -1, lo que indica que la cola inicia vacía.

La función estaVacia verifica si no hay elementos en la cola comparando si final < frente. Esto ocurre cuando no se han insertado elementos o cuando todos han sido eliminados.

La función estaLlena comprueba si la cola ha alcanzado su capacidad máxima (final == MAX - 1).

El método enqueue agrega un elemento al final de la cola, incrementando el índice final. Antes de insertar, valida que la cola no esté llena.

El método dequeue elimina y devuelve el elemento ubicado en el frente de la cola, incrementando el índice frente. Si la cola está vacía, muestra un mensaje de error y devuelve -1.

La función mostrar recorre la cola desde frente hasta final, imprimiendo los elementos en el orden en que serán atendidos, respetando el comportamiento FIFO.

Finalmente, tamaño calcula la cantidad de elementos en la cola mediante la expresión final - frente + 1, retornando 0 si la cola está vacía.

Un aspecto importante es que esta implementación no reutiliza espacios liberados (no es una cola circular), por lo que después de varios dequeue, aunque haya espacio disponible al inicio del arreglo, no se podrá volver a utilizar.

```
1 #include <stdio.h>
2 #include "cola.h"
3
4 void inicializar(Cola *c) {
5     c->frente = 0;
6     c->final = -1;
7 }
8
9 int estaVacia(Cola *c) {
10     return c->final < c->frente;
11 }
12
13 int estaLlena(Cola *c) {
14     return c->final == MAX - 1;
15 }
16
17 void enqueue(Cola *c, int valor) {
18     if (!estaLlena(c)) {
19         c->datos[++c->final] = valor;
20         printf("Elemento agregado\n");
21     } else {
22         printf("Cola llena\n");
23     }
24 }
25
26 int dequeue(Cola *c) {
27     if (!estaVacia(c)) {
28         return c->datos[c->frente++];
29     } else {
30         printf("Cola vacia\n");
31         return -1;
32     }
33 }
34
35 void mostrar(Cola *c) {
36     if (estaVacia(c)) {
37         printf("Cola vacia\n");
38         return;
39     }
40
41     for (int i = c->frente; i <= c->final; i++) {
42         printf("%d\n", c->datos[i]);
43     }
44 }
```

```
45
46 int tamaño(Cola *c) {
47     if (estaVacia(c)) {
48         return 0;
49     }
50     return c->final - c->frente + 1;
51 }
```

Ordenamientos.h

El archivo Ordenamientos.h es un encabezado que declara los prototipos de tres algoritmos de ordenamiento: Burbuja, QuickSort y MergeSort. Su función es permitir que estos métodos puedan ser utilizados desde otros archivos del programa sin necesidad de conocer su implementación interna.

Cada función trabaja con arreglos de tipo entero. En el caso de burbuja, se recibe el arreglo junto con su tamaño, mientras que quickSort y mergeSort reciben índices que delimitan la parte del arreglo que se desea ordenar, lo cual permite aplicar recursividad.

Este archivo únicamente define la interfaz de uso de los algoritmos, sin incluir su lógica interna.

```
1 #ifndef ORDENAMIENTOS_H
2 #define ORDENAMIENTOS_H
3
4 void burbuja(int arr[], int n);
5 void quickSort(int arr[], int low, int high);
6 void mergeSort(int arr[], int left, int right);
7
8 #endif
```

Ordenamientos.cpp

El archivo Ordenamientos.cpp contiene la implementación de tres algoritmos de ordenamiento ampliamente utilizados.

El método Burbuja recorre el arreglo múltiples veces comparando elementos adyacentes y los intercambia si están en desorden. Es sencillo de implementar, pero su eficiencia es baja en arreglos grandes.

El algoritmo QuickSort utiliza la técnica de “divide y vencerás”. Selecciona un pivote (último elemento del arreglo), reorganiza los elementos menores a la izquierda y los mayores a la derecha mediante la función particion, y luego aplica el mismo proceso de forma recursiva a los subarreglos generados.

El algoritmo MergeSort también sigue el enfoque de “divide y vencerás”. Divide el arreglo en partes más pequeñas hasta llegar a elementos individuales y luego los combina ordenadamente mediante la función merge, utilizando un arreglo auxiliar.

Un detalle importante es que la función merge utiliza un arreglo temporal de tamaño fijo (temp[50]), lo que limita el tamaño máximo del arreglo que se puede ordenar correctamente. Para un uso más general, este tamaño debería ajustarse dinámicamente.

En conjunto, estos algoritmos permiten comparar diferentes enfoques de ordenamiento, desde uno simple (Burbuja) hasta métodos más eficientes (QuickSort y MergeSort).

```
1 #include "ordenamientos.h"
2
3 // Burbuja
4 void burbuja(int arr[], int n) {
5     for (int i = 0; i < n - 1; i++) {
6         for (int j = 0; j < n - i - 1; j++) {
7             if (arr[j] > arr[j + 1]) {
8                 int temp = arr[j];
9                 arr[j] = arr[j + 1];
10                arr[j + 1] = temp;
11            }
12        }
13    }
14 }
15
16 // QuickSort
17 int particion(int arr[], int low, int high) {
18     int pivote = arr[high];
19     int i = low - 1;
20
21     for (int j = low; j < high; j++) {
22         if (arr[j] < pivote) {
23             i++;
24             int temp = arr[i];
25             arr[i] = arr[j];
26             arr[j] = temp;
27         }
28     }
29
30     int temp = arr[i + 1];
31     arr[i + 1] = arr[high];
32     arr[high] = temp;
33
34     return i + 1;
35 }
36
37 void quickSort(int arr[], int low, int high) {
38     if (low < high) {
39         int pi = particion(arr, low, high);
40         quickSort(arr, low, pi - 1);
41         quickSort(arr, pi + 1, high);
42     }
43 }
```

```

44
45 // MergeSort
46 void merge(int arr[], int left, int mid, int right) {
47     int temp[50];
48     int i = left;
49     int j = mid + 1;
50     int k = 0;
51
52     while (i <= mid && j <= right) {
53         if (arr[i] < arr[j]) temp[k++] = arr[i++];
54         else temp[k++] = arr[j++];
55     }
56
57     while (i <= mid) temp[k++] = arr[i++];
58     while (j <= right) temp[k++] = arr[j++];
59
60     for (i = left, k = 0; i <= right; i++, k++) {
61         arr[i] = temp[k];
62     }
63 }
64
65 void mergeSort(int arr[], int left, int right) {
66     if (left < right) {
67         int mid = (left + right) / 2;
68         mergeSort(arr, left, mid);
69         mergeSort(arr, mid + 1, right);
70         merge(arr, left, mid, right);
71     }
72 }

```

Clase main.cpp

El archivo main.cpp contiene el programa principal que permite interactuar con la estructura de datos cola y aplicar algoritmos de ordenamiento sobre sus elementos.

Se incluyen las bibliotecas necesarias: stdio.h para entrada y salida de datos, cola.h para manejar la estructura de la cola y ordenamientos.h para utilizar los métodos de ordenamiento.

Al inicio, se declara una variable de tipo Cola y se inicializa con la función inicializar, dejando la cola vacía (frente = 0, final = -1).

El programa funciona mediante un menú interactivo dentro de un ciclo do-while, que se repite hasta que el usuario selecciona la opción de salir. Las opciones disponibles permiten:

- Insertar elementos en la cola (enqueue), agregándolos al final.
- Eliminar elementos (dequeue), retirándolos desde el frente, respetando el principio FIFO (First In, First Out).
- Mostrar los elementos en el orden en que serán atendidos.
- Ordenar los elementos de la cola utilizando tres algoritmos: Burbuja, QuickSort y MergeSort.

Para los métodos de ordenamiento:

- En Burbuja, se utiliza el arreglo interno junto con el tamaño de la cola.
- En QuickSort y MergeSort, se emplean los índices frente y final para ordenar únicamente la parte ocupada del arreglo.

Un aspecto importante es que, al ordenar directamente el arreglo interno, se modifica el orden original de la cola, lo cual rompe el comportamiento FIFO. Es decir, después de ordenar, los elementos ya no se comportan estrictamente como una cola.

Además, no se valida si la cola está vacía antes de aplicar los algoritmos, lo que podría provocar comportamientos incorrectos si no hay elementos.

```
1 #include <stdio.h>
2 #include "cola.h"
3 #include "ordenamientos.h"
4
5 int main() {
6     Cola c;
7     inicializar(&c);
8
9     int opcion, num;
10
11     do {
12         printf("\n----- MENU ----- \n");
13         printf("1. Enqueue\n");
14         printf("2. Dequeue\n");
15         printf("3. Mostrar\n");
16         printf("4. Ordenar Burbuja\n");
17         printf("5. Ordenar QuickSort\n");
18         printf("6. Ordenar MergeSort\n");
19         printf("7. Salir\n");
20         printf("Opcion: ");
21         scanf("%d", &opcion);
22
23         switch(opcion) {
24             case 1:
25                 printf("Numero: ");
26                 scanf("%d", &num);
27                 enqueue(&c, num);
28                 break;
29
30             case 2:
31                 printf("Eliminado: %d\n", dequeue(&c));
32                 break;
33
34             case 3:
35                 mostrar(&c);
36                 break;
37
38             case 4:
39                 burbuja(c.datos, tamano(&c));
40                 printf("Ordenado con Burbuja\n");
41                 break;
42
43             case 5:
44                 quickSort(c.datos, c.frente, c.final);
45                 printf("Ordenado con QuickSort\n");
46                 break;
47
48             case 6:
49                 mergeSort(c.datos, c.frente, c.final);
50                 printf("Ordenado con MergeSort\n");
51                 break;
52
53             case 7:
54                 printf("Saliendo...\n");
55                 break;
56
57             default:
58                 printf("Opcion invalida\n");
59         }
60     } while(opcion != 7);
61
62     return 0;
63 }
64 }
```

Ejecución del programa:

Iniciamos la prueba insertando elementos a la cola mediante la opción 1 (Enqueue).
Se agregaron los siguientes valores:

1, 16, 54, 32, y 29

```
----- MENU -----
1. Enqueue
2. Dequeue
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 1
Numero: 1
Elemento agregado

----- MENU -----
1. Enqueue
2. Dequeue
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 1
Numero: 16
Elemento agregado
```

```
----- MENU -----
1. Enqueue
2. Dequeue
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 1
Numero: 29
Elemento agregado
```

```
----- MENU -----
1. Enqueue
2. Dequeue
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 1
Numero: 54
Elemento agregado

----- MENU -----
1. Enqueue
2. Dequeue
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 1
Numero: 32
Elemento agregado
```

Al seleccionar la opción 3 (Mostrar), se imprimen los datos en el siguiente orden:

1, 16, 54, 32, 29

Esto confirma el comportamiento FIFO (First In, First Out) de la cola, ya que los elementos se muestran en el mismo orden en que fueron insertados.

```
----- MENU -----
1. Enqueue
2. Dequeue
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 3
1
16
54
32
29
```

Posteriormente, se aplica el ordenamiento con la opción 4 (Burbuja), obteniendo:

1, 16, 29, 32, 54

El arreglo interno de la cola se ordena de menor a mayor, y al mostrarse mantiene ese orden.

```
----- MENU -----
1. Enqueue
2. Dequeue
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 4
Ordenado con Burbuja
1
16
29
32
54
```

Las opciones 5 (QuickSort) y 6 (MergeSort) producen exactamente el mismo resultado ordenado:

1, 16, 29, 32, 54

```
----- MENU -----
1. Enqueue
2. Dequeue
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 5
Ordenado con QuickSort
1
16
29
32
54

----- MENU -----
1. Enqueue
2. Dequeue
3. Mostrar
4. Ordenar Burbuja
5. Ordenar QuickSort
6. Ordenar MergeSort
7. Salir
Opcion: 6
Ordenado con MergeSort
1
16
29
32
54
```

Después, se utiliza la opción 2 (Dequeue), eliminando el primer elemento de la cola:

Eliminado: 1

Esto confirma el comportamiento FIFO, ya que el primer elemento insertado es el primero en salir.

```
----- MENU -----  
1. Enqueue  
2. Dequeue  
3. Mostrar  
4. Ordenar Burbuja  
5. Ordenar QuickSort  
6. Ordenar MergeSort  
7. Salir  
Opcion: 2  
Eliminado: 1
```

Finalmente, al mostrar nuevamente la cola (opción 3), se obtiene:

16, 29, 32, 54

Lo cual refleja correctamente el estado actual tras la eliminación.

```
----- MENU -----  
1. Enqueue  
2. Dequeue  
3. Mostrar  
4. Ordenar Burbuja  
5. Ordenar QuickSort  
6. Ordenar MergeSort  
7. Salir  
Opcion: 3  
16  
29  
32  
54
```

Ejercicio 5: Lista

Descripción del programa:

El programa implementa una lista estática utilizando un arreglo en lenguaje C++.

A diferencia de la pila y la cola, la lista permite manejar los elementos con mayor flexibilidad, ya que no está limitada por un orden específico como LIFO o FIFO.

El sistema funciona mediante un menú en consola que permite al usuario realizar diferentes operaciones sobre la lista, tales como:

- Insertar elementos
- Eliminar elementos
- Buscar elementos
- Mostrar la lista
- Ordenar los datos mediante:
 - Burbuja
 - QuickSort
 - MergeSort

La lista utiliza memoria estática, es decir, el tamaño del arreglo está definido desde el inicio y no cambia durante la ejecución del programa.

Descripción de clases:

Clase Lista.h

El archivo Lista.h es un encabezado que define la estructura y operaciones de una lista estática implementada mediante un arreglo.

Se utilizan directivas de preprocesador (`#ifndef`, `#define`, `#endif`) para evitar múltiples inclusiones durante la compilación.

Se define la constante MAX con valor 50, que representa la capacidad máxima de la lista.

La estructura Lista contiene:

- Un arreglo datos donde se almacenan los elementos.
- Una variable tam que indica la cantidad actual de elementos en la lista.

A diferencia de la pila y la cola, la lista permite acceder y manipular elementos en cualquier posición, lo que la hace más flexible.

También se declaran funciones para:

- Inicializar la lista
- Insertar y eliminar elementos

- Buscar un valor
- Mostrar el contenido
- Verificar si está llena o vacía

Este archivo solo define la interfaz; la lógica se implementa en otro archivo.

```

1  #ifndef LISTA_H
2  #define LISTA_H
3
4  #define MAX 50
5
6  typedef struct {
7      int datos[MAX];
8      int tam;
9  } Lista;
10
11 void inicializar(Lista *l);
12 void insertar(Lista *l, int valor);
13 void eliminar(Lista *l, int valor);
14 int buscar(Lista *l, int valor);
15 void mostrar(Lista *l);
16 int estaLlena(Lista *l);
17 int estaVacia(Lista *l);
18
19 #endif

```

Clase Lista.cpp

El archivo Lista.cpp contiene la implementación de las operaciones de la lista.

La función inicializar establece tam = 0, indicando que la lista comienza vacía.

Las funciones estaLlena y estaVacia permiten verificar el estado de la lista comparando el tamaño actual con sus límites.

El método insertar agrega un elemento al final de la lista si hay espacio disponible, incrementando el tamaño.

La función buscar recorre la lista secuencialmente para encontrar un valor y devuelve su posición; si no lo encuentra, retorna -1.

El método eliminar primero busca la posición del elemento. Si existe, desplaza todos los elementos posteriores una posición hacia la izquierda para mantener el orden y luego reduce el tamaño de la lista.

La función mostrar imprime todos los elementos desde la primera posición hasta el último elemento almacenado.

Esta implementación mantiene los elementos ordenados según su inserción (no necesariamente ordenados numéricamente), y cualquier eliminación conserva la continuidad del arreglo.

```
1 #include <stdio.h>
2 #include "lista.h"
3
4 void inicializar(Lista *l) {
5     l->tam = 0;
6 }
7
8 int estaLlena(Lista *l) {
9     return l->tam == MAX;
10 }
11
12 int estaVacia(Lista *l) {
13     return l->tam == 0;
14 }
15
16 void insertar(Lista *l, int valor) {
17     if (!estaLlena(l)) {
18         l->datos[l->tam++] = valor;
19         printf("Elemento insertado\n");
20     } else {
21         printf("Lista llena\n");
22     }
23 }
24
25 int buscar(Lista *l, int valor) {
26     for (int i = 0; i < l->tam; i++) {
27         if (l->datos[i] == valor) {
28             return i;
29         }
30     }
31     return -1;
32 }
33
34 void eliminar(Lista *l, int valor) {
35     int pos = buscar(l, valor);
36
37     if (pos == -1) {
38         printf("Elemento no encontrado\n");
39         return;
40     }
41
42     for (int i = pos; i < l->tam - 1; i++) {
43         l->datos[i] = l->datos[i + 1];
44     }
```

```

45
46     l->tam--;
47     printf("Elemento eliminado\n");
48 }
49
50 void mostrar(Lista *l) {
51     if (estaVacia(l)) {
52         printf("Lista vacia\n");
53         return;
54     }
55
56     for (int i = 0; i < l->tam; i++) {
57         printf("%d\n", l->datos[i]);
58     }
59 }

```

Clase Ordenamientos.h

El archivo Ordenamientos.h es un encabezado que declara los prototipos de tres algoritmos de ordenamiento: Burbuja, Quick Sort y Merge Sort. Su función es permitir que estos métodos puedan ser utilizados desde otros archivos del programa sin necesidad de conocer su implementación interna.

Cada función trabaja con arreglos de enteros. La función burbuja recibe el arreglo y su tamaño, mientras que quickSort y mergeSort utilizan índices (inicio y fin) para poder aplicar recursividad sobre subarreglos.

Este archivo define únicamente la interfaz de los algoritmos, sin incluir su lógica.

```

1 #ifndef ORDENAMIENTOS_H
2 #define ORDENAMIENTOS_H
3
4 void burbuja(int arr[], int n);
5 void quickSort(int arr[], int low, int high);
6 void mergeSort(int arr[], int left, int right);
7
8 #endif

```

Clase Ordenamientos.cpp

El archivo Ordenamientos.cpp contiene la implementación de tres algoritmos de ordenamiento clásicos.

El algoritmo Burbuja compara elementos adyacentes y los intercambia si están en el orden incorrecto. Repite este proceso varias veces hasta que el arreglo queda ordenado. Es fácil de entender, pero poco eficiente en arreglos grandes.

El algoritmo Quick Sort utiliza la técnica de “divide y vencerás”. Selecciona un pivote (en este caso, el último elemento), reorganiza el arreglo colocando los valores menores a la izquierda y los mayores a la derecha mediante la función particion, y luego aplica el mismo proceso de forma recursiva.

El algoritmo Merge Sort también divide el arreglo en partes más pequeñas hasta llegar a elementos individuales, y posteriormente los combina de forma ordenada utilizando la función merge y un arreglo auxiliar.

Un detalle importante es que en merge se usa un arreglo temporal de tamaño fijo (temp[50]), lo que limita el tamaño máximo de los datos que se pueden ordenar correctamente.

En conjunto, estos algoritmos permiten comparar diferentes enfoques de ordenamiento: uno simple (Burbuja) y dos más eficientes (Quick Sort y Merge Sort).

```
1 #include "ordenamientos.h"
2
3 // Burbuja
4 void burbuja(int arr[], int n) {
5     for (int i = 0; i < n - 1; i++) {
6         for (int j = 0; j < n - i - 1; j++) {
7             if (arr[j] > arr[j + 1]) {
8                 int temp = arr[j];
9                 arr[j] = arr[j + 1];
10                arr[j + 1] = temp;
11            }
12        }
13    }
14 }
15
16 // Quick Sort
17 int particion(int arr[], int low, int high) {
18     int pivote = arr[high];
19     int i = low - 1;
20
21     for (int j = low; j < high; j++) {
22         if (arr[j] < pivote) {
23             i++;
24             int temp = arr[i];
25             arr[i] = arr[j];
26             arr[j] = temp;
27         }
28     }
29
30     int temp = arr[i + 1];
31     arr[i + 1] = arr[high];
32     arr[high] = temp;
33
34     return i + 1;
35 }
36
37 void quickSort(int arr[], int low, int high) {
38     if (low < high) {
39         int pi = particion(arr, low, high);
40         quickSort(arr, low, pi - 1);
41         quickSort(arr, pi + 1, high);
42     }
43 }
```

```

44
45 // Merge Sort
46 void merge(int arr[], int left, int mid, int right) {
47     int temp[50];
48     int i = left, j = mid + 1, k = 0;
49
50     while (i <= mid && j <= right) {
51         if (arr[i] < arr[j]) temp[k++] = arr[i++];
52         else temp[k++] = arr[j++];
53     }
54
55     while (i <= mid) temp[k++] = arr[i++];
56     while (j <= right) temp[k++] = arr[j++];
57
58     for (i = left, k = 0; i <= right; i++, k++) {
59         arr[i] = temp[k];
60     }
61 }
62
63 void mergeSort(int arr[], int left, int right) {
64     if (left < right) {
65         int mid = (left + right) / 2;
66         mergeSort(arr, left, mid);
67         mergeSort(arr, mid + 1, right);
68         merge(arr, left, mid, right);
69     }
70 }

```

Clase main.cpp

El archivo main.cpp contiene el programa principal que permite interactuar con la estructura de datos lista y aplicar algoritmos de ordenamiento sobre sus elementos.

Se incluyen las bibliotecas necesarias: stdio.h para la entrada y salida de datos, lista.h para manejar la estructura de la lista y ordenamientos.h para utilizar los métodos de ordenamiento.

Al inicio del programa se declara una variable de tipo Lista y se inicializa mediante la función inicializar, dejando la lista vacía (tam = 0).

El programa funciona mediante un menú interactivo dentro de un ciclo do-while, el cual se repite hasta que el usuario selecciona la opción de salir. Las opciones disponibles permiten:

- Insertar elementos en la lista, agregándolos al final.
- Eliminar un elemento específico, buscándolo y desplazando los elementos posteriores para mantener el orden.
- Buscar un valor dentro de la lista, devolviendo su posición si existe.
- Mostrar todos los elementos almacenados.

Además, se incluyen opciones para ordenar la lista utilizando tres algoritmos:

- Burbuja, que ordena comparando elementos adyacentes.
- QuickSort, que utiliza partición y recursividad.
- MergeSort, que divide el arreglo y lo combina ordenadamente.

Para los ordenamientos, se trabaja directamente con el arreglo interno (l.datos) y el tamaño actual (l.tam). Después de ordenar, se muestran los elementos para verificar el resultado.

Un aspecto importante es que los algoritmos modifican directamente el contenido de la lista, por lo que el orden original de inserción se pierde después de aplicar cualquier método de ordenamiento.

```
1 #include <stdio.h>
2 #include "lista.h"
3 #include "ordenamientos.h"
4
5 int main() {
6     lista l;
7     inicializar(&l);
8
9     int opcion, num;
10
11     do {
12         printf("\n--- MENU LISTA ---\n");
13         printf("1. Insertar\n");
14         printf("2. Eliminar\n");
15         printf("3. Buscar\n");
16         printf("4. Mostrar\n");
17         printf("5. Ordenar Burbuja\n");
18         printf("6. Ordenar QuickSort\n");
19         printf("7. Ordenar MergeSort\n");
20         printf("8. Salir\n");
21         printf("Opcion: ");
22         scanf("%d", &opcion);
23
24         switch(opcion) {
25             case 1:
26                 printf("Numero: ");
27                 scanf("%d", &num);
28                 insertar(&l, num);
29                 break;
30
31             case 2:
32                 printf("Numero a eliminar: ");
33                 scanf("%d", &num);
34                 eliminar(&l, num);
35                 break;
36
37             case 3:
38                 printf("Numero a buscar: ");
39                 scanf("%d", &num);
40                 int pos = buscar(&l, num);
41                 if (pos != -1)
42                     printf("Encontrado en posicion %d\n", pos);
```

```

43         else
44             printf("No encontrado\n");
45             break;
46
47         case 4:
48             mostrar(&l);
49             break;
50
51         case 5:
52             burbuja(l.datos, l.tam);
53             printf("Ordenado con Burbuja\n");
54             mostrar(&l);
55             break;
56
57         case 6:
58             quickSort(l.datos, 0, l.tam - 1);
59             printf("Ordenado con QuickSort\n");
60             mostrar(&l);
61             break;
62
63         case 7:
64             mergeSort(l.datos, 0, l.tam - 1);
65             printf("Ordenado con MergeSort\n");
66             mostrar(&l);
67             break;
68
69         case 8:
70             printf("Salir\n");
71             break;
72
73         default:
74             printf("Opcion invalida\n");
75     }
76
77     } while(opcion != 8);
78
79     return 0;
80 }

```

Ejecución del programa:

Se insertan elementos mediante la opción 1 (Insertar) en el siguiente orden:

1, 16, 54, 32, 29

```

--- MENU LISTA ---
1. Insertar
2. Eliminar
3. Buscar
4. Mostrar
5. Ordenar Burbuja
6. Ordenar QuickSort
7. Ordenar MergeSort
8. Salir
Opcion: 1
Numero: 1
Elemento insertado

```

```

--- MENU LISTA ---
1. Insertar
2. Eliminar
3. Buscar
4. Mostrar
5. Ordenar Burbuja
6. Ordenar QuickSort
7. Ordenar MergeSort
8. Salir
Opcion: 1
Numero: 16
Elemento insertado

```

```
--- MENU LISTA ---
1. Insertar
2. Eliminar
3. Buscar
4. Mostrar
5. Ordenar Burbuja
6. Ordenar QuickSort
7. Ordenar MergeSort
8. Salir
Opcion: 1
Numero: 54
Elemento insertado
```

```
--- MENU LISTA ---
1. Insertar
2. Eliminar
3. Buscar
4. Mostrar
5. Ordenar Burbuja
6. Ordenar QuickSort
7. Ordenar MergeSort
8. Salir
Opcion: 1
Numero: 32
Elemento insertado
```

```
--- MENU LISTA ---
1. Insertar
2. Eliminar
3. Buscar
4. Mostrar
5. Ordenar Burbuja
6. Ordenar QuickSort
7. Ordenar MergeSort
8. Salir
Opcion: 1
Numero: 29
Elemento insertado
```

Al seleccionar la opción 4(Mostrar), se obtiene:

1, 16, 54, 32, 29

Lo cual confirma que la lista conserva el orden de inserción.

```
--- MENU LISTA ---
1. Insertar
2. Eliminar
3. Buscar
4. Mostrar
5. Ordenar Burbuja
6. Ordenar QuickSort
7. Ordenar MergeSort
8. Salir
Opcion: 4
1
16
54
32
29
```

Posteriormente, se utiliza la opción 3 (Buscar):

Al buscar el valor 1, se encuentra en la posición 0.

Al buscar el valor 54, se encuentra en la posición 2.

Esto demuestra que la función de búsqueda recorre correctamente la lista y devuelve la posición del elemento.

```
--- MENU LISTA ---
1. Insertar
2. Eliminar
3. Buscar
4. Mostrar
5. Ordenar Burbuja
6. Ordenar QuickSort
7. Ordenar MergeSort
8. Salir
Opcion: 3
Numero a buscar: 1
Encontrado en posicion 0

--- MENU LISTA ---
1. Insertar
2. Eliminar
3. Buscar
4. Mostrar
5. Ordenar Burbuja
6. Ordenar QuickSort
7. Ordenar MergeSort
8. Salir
Opcion: 3
Numero a buscar: 54
Encontrado en posicion 2
```

Después se aplica el ordenamiento con la opción 5 (Burbuja), obteniendo:
1, 16, 29, 32, 54

```
--- MENU LISTA ---
1. Insertar
2. Eliminar
3. Buscar
4. Mostrar
5. Ordenar Burbuja
6. Ordenar QuickSort
7. Ordenar MergeSort
8. Salir
Opcion: 5
Ordenado con Burbuja
1
16
29
32
54
```

Los elementos quedan organizados de menor a mayor. Las opciones 6 (QuickSort) y 7 (MergeSort) generan exactamente el mismo resultado, lo que confirma la correcta implementación de los tres algoritmos.

```
--- MENU LISTA ---
1. Insertar
2. Eliminar
3. Buscar
4. Mostrar
5. Ordenar Burbuja
6. Ordenar QuickSort
7. Ordenar MergeSort
8. Salir
Opcion: 6
Ordenado con QuickSort
1
16
29
32
54
```

```
--- MENU LISTA ---
1. Insertar
2. Eliminar
3. Buscar
4. Mostrar
5. Ordenar Burbuja
6. Ordenar QuickSort
7. Ordenar MergeSort
8. Salir
Opcion: 7
Ordenado con MergeSort
1
16
29
32
54
```

Luego, se elimina el valor 54 mediante la opción 2 (Eliminar), mostrando:

Elemento eliminado

```
--- MENU LISTA ---  
1. Insertar  
2. Eliminar  
3. Buscar  
4. Mostrar  
5. Ordenar Burbuja  
6. Ordenar QuickSort  
7. Ordenar MergeSort  
8. Salir  
Opcion: 2  
Numero a eliminar: 54  
Elemento eliminado
```

Finalmente, al mostrar la lista (opción 4), se obtiene:

1, 16, 29, 32

Lo que confirma que el elemento fue eliminado correctamente y que los elementos restante se reorganizaron manteniendo la continuidad del arreglo.

```
--- MENU LISTA ---  
1. Insertar  
2. Eliminar  
3. Buscar  
4. Mostrar  
5. Ordenar Burbuja  
6. Ordenar QuickSort  
7. Ordenar MergeSort  
8. Salir  
Opcion: 4  
1  
16  
32  
29
```



Instituto Politécnico Nacional
Unidad Profesional
Interdisciplinaria de Ingeniería y
Ciencias Sociales y
Administrativas



Prácticas

Segundo Parcial

Integrantes

Pedro Zenón Monserrat

No. Lista

32

Profesor: Yventz Entzana Garduño

Secuencia: 2NM23

Repositorio de GitHub:

<https://github.com/Monserrat2007/programa-ordenamiento>

Ordenamiento burbuja ordenar conjunto de caracteres, enteros y nuevo tipo de dato (PE y POO)

Introducción: En este proyecto se desarrolló un programa basado en el algoritmo de ordenamiento Burbuja, aplicado a tres tipos de datos: enteros, caracteres y un nuevo tipo definido como *Persona* (nombre y edad). El propósito fue implementar este algoritmo utilizando programación orientada a objetos, integrando estructuras dinámicas y respetando una organización clara del código. Además, se buscó comprender la diferencia entre un ordenamiento directo e indirecto, aplicando correctamente cada enfoque según lo solicitado.

Objetivo: Desarrollar un programa que implemente el algoritmo de ordenamiento Burbuja para distintos tipos de datos, utilizando programación orientada a objetos, estructuras dinámicas y aplicando el concepto de ordenamiento indirecto mediante índices.

Desarrollo: Se implementó un sistema que permite ingresar datos de tipo entero, carácter y una estructura llamada *Persona*, que contiene nombre y edad. Para cumplir con los requerimientos, se utilizó una clase abstracta encargada de la carga de datos y una interfaz que define el método de ordenamiento. El algoritmo Burbuja se aplicó de forma **indirecta**, utilizando vectores de índices (idx) que permiten ordenar sin modificar directamente los datos originales. Se emplearon estructuras dinámicas (vector) para almacenar la información, lo que permite mayor flexibilidad en comparación con arreglos estáticos.

El programa sigue un flujo simple:

1. Carga de datos
2. Ordenamiento mediante Burbuja
3. Visualización de resultados ordenados

Errores encontrados: Durante el desarrollo del programa se identificaron los siguientes errores:

- Inicialmente se implementó el ordenamiento de forma directa, intercambiando los datos en lugar de los índices.
- Se utilizaron arreglos estáticos en lugar de estructuras dinámicas (vector).
- No se incluía una clase abstracta ni una interfaz, lo cual incumplía los requisitos del profesor.
- Se implementó un menú innecesario que complicaba la ejecución del programa.
- Existían problemas en la organización del código, separando la lógica en múltiples clases sin una estructura clara.

Soluciones aplicadas

Para corregir los errores detectados se realizaron las siguientes mejoras:

- Se implementó correctamente el ordenamiento indirecto utilizando vectores de índices (idx).
- Se reemplazaron los arreglos estáticos por vector, permitiendo manejo dinámico de datos.
- Se agregó una interfaz para definir el método de ordenamiento (burbuja).
- Se incorporó una clase abstracta para la carga de datos, mejorando la estructura del programa.

- Se eliminó el menú, simplificando el flujo de ejecución.
- Se unificó la lógica en una sola clase organizada, facilitando el mantenimiento del código.

Resultados: El programa final funciona correctamente, permitiendo ordenar conjuntos de enteros, caracteres y estructuras tipo Persona mediante el algoritmo Burbuja. Se logró aplicar de forma adecuada el ordenamiento indirecto, manteniendo los datos originales sin modificación directa. Además, el programa cumple con los principios básicos de programación orientada a objetos y con los requisitos establecidos, logrando un código más limpio, estructurado y funcional.

```

100 void ordenarPersonas() {
101     for (size_t i = 0; i < idxPersonas.size() - 1; i++) {
102         for (size_t j = 0; j < idxPersonas.size() - i - 1; j++) {
103             if (personas[idxPersonas[j]].edad > personas[idxPersonas[j + 1]].edad) {
104                 swap(idxPersonas[j], idxPersonas[j + 1]);
105             }
106         }
107     }
108 }
109
110 void mostrar() {
111     cout << "\nEnteros ordenados:\n";
112     for (int i : idxEnteros)
113         cout << enteros[i] << " ";
114
115     cout << "\nCaracteres ordenados:\n";
116     for (int i : idxCaracteres)
117         cout << caracteres[i] << " ";
118
119     cout << "\n\nPersonas ordenadas:\n";
120     for (int i : idxPersonas)
121         cout << personas[i].nombre << " - " << personas[i].edad << endl;
122 }
123
124 // ===== M&A;T&N =====
125

```

Línea	Col.	Archivo	Mensaje
42	22	C:\Users\Monserat 2007\Downloads\programas corregui...	[Warning] override controls (override/final) only available with -std=c++11 or -std=gnu++11
74	18	C:\Users\Monserat 2007\Downloads\programas corregui...	[Warning] override controls (override/final) only available with -std=c++11 or -std=gnu++11
		C:\Users\Monserat 2007\Downloads\programas corre...	In member function 'void Ordenamiento::mostrar()':
112	22	C:\Users\Monserat 2007\Downloads\programas corregui...	[Error] range-based 'for' loops are not allowed in C++98 mode
116	22	C:\Users\Monserat 2007\Downloads\programas corregui...	[Error] range-based 'for' loops are not allowed in C++98 mode
120	22	C:\Users\Monserat 2007\Downloads\programas corregui...	[Error] range-based 'for' loops are not allowed in C++98 mode

```

C:\Users\Monserat 2007\Do...
Cuantos enteros: 3
8
9
8
Cuantos caracteres: 4
7
i
o
p
Cuantas personas: 3
Nombre: monse
Edad: 19
Nombre: ulises
Edad: 18
Nombre: cristian
Edad: 18
Enteros ordenados:
8 8 9
Caracteres ordenados:
7 i o p
Personas ordenadas por edad:
ulises - 18
cristian - 18
monse - 19
=====
Process exited after 29.94 seconds with return value 0

```

Ordenamiento merge sort ordenar conjunto de caracteres, enteros y nuevo tipo de dato (PE y POO)

Introducción: En este programa trabajé con el algoritmo de ordenamiento MergeSort, aplicándolo a tres tipos de datos: enteros, caracteres y un tipo de dato creado llamado *Persona*, que incluye nombre y edad. La idea fue hacer todo usando programación orientada a objetos y estructuras dinámicas, además de aplicar el ordenamiento indirecto, que fue lo que más se pidió. MergeSort es un algoritmo más avanzado porque no ordena directamente, sino que divide los datos y luego los va uniando de forma ordenada, lo cual lo hace más eficiente.

Objetivo: Desarrollar un programa que implemente el algoritmo MergeSort para ordenar enteros, caracteres y personas, usando programación orientada a objetos y aplicando el ordenamiento indirecto con índices.

Desarrollo: Primero se pidió al usuario ingresar datos: números, caracteres y personas con nombre y edad. Después, esos datos se guardaron en vectores para que fueran dinámicos. El ordenamiento se hizo de forma indirecta, usando vectores de índices (idx), en lugar de mover los datos directamente. Esto significa que los datos originales no cambian de lugar, solo se reorganizan sus posiciones mediante índices. También se usó una clase abstracta para la carga de datos y una interfaz para definir el método de ordenamiento (mergesort), para cumplir con la estructura que pidió el profesor. El algoritmo funciona dividiendo los datos en partes pequeñas, ordenándolas y luego uniéndolas otra vez ya ordenadas.

Errores encontrados

Durante el desarrollo tuve varios errores:

- Al inicio confundí el algoritmo y no estaba usando MergeSort correctamente.
- Estaba ordenando directamente los datos en lugar de hacerlo de forma indirecta.
- No usaba bien el proceso de mezcla (merge), lo que hacía que el resultado fuera incorrecto.
- Usé arreglos normales en lugar de vectores dinámicos.
- No había interfaz ni clase abstracta al principio.
- También había agregado un menú que realmente no era necesario.

Soluciones aplicadas

Para corregir todo eso:

- Implementé correctamente MergeSort usando recursividad.
- Cambié el ordenamiento a forma indirecta usando índices (idx).
- Arreglé la parte del merge para que combinara bien los datos.
- Reemplacé los arreglos por vector.
- Agregué la interfaz y la clase abstracta como se pedía.
- Eliminé el menú para que el programa fuera más simple.

Resultados: Al final, el programa funciona bien y ordena correctamente los tres tipos de datos usando MergeSort. Se logró aplicar el ordenamiento indirecto sin modificar los datos originales, solo usando índices. También se cumplió con todo lo que pidió el profesor, tanto en funcionalidad como en estructura del código.

```
C:\Users\Monserrat 2007\Downloads\programas corregidos\Ordenamiento POO merge sort.cpp - [Executing] - Dev-C++ 5.11
C:\Users\Monserrat 2007\Downloads\programas corregidos\Ordenamiento POO merge sort.cpp - [Executing] - Dev-C++ 5.11
Cuantos caracteres: 5
a
r
g
t
m
Cuantas personas: 4
Nombre: monse 89
Edad: Nombre: ulises
Edad: 39
Nombre: juan
Edad: 19
Nombre: erik
Edad: 18

Enteros:
7 14 90

Caracteres:
a g m r t

Personas:
erik - 18
juan - 19
ulises - 39
monse - 89

-----
Process exited after 27.1 seconds with return value 0
Presione una tecla para continuar . . .
```

Ordenamiento quick sort ordenar conjunto de caracteres, enteros y nuevo tipo de dato (PE y POO)

Introducción: En este programa trabajé con el algoritmo de ordenamiento QuickSort, aplicándolo a tres tipos de datos: enteros, caracteres y un tipo de dato definido como *Persona* (nombre y edad). La intención fue implementar este algoritmo usando programación orientada a objetos, estructuras dinámicas y el método de ordenamiento indirecto. QuickSort es un algoritmo eficiente que funciona seleccionando un pivote y acomodando los elementos menores y mayores alrededor de él, para después ordenar las partes de forma recursiva.

Objetivo: Desarrollar un programa que implemente el algoritmo QuickSort para ordenar diferentes tipos de datos (enteros, caracteres y personas), utilizando programación orientada a objetos y ordenamiento indirecto mediante índices.

Desarrollo: Primero se le pidió al usuario ingresar los datos: números, caracteres y personas con nombre y edad. Estos datos se guardaron en vectores para que fueran dinámicos. El ordenamiento se hizo de forma indirecta, usando vectores de índices (idx), lo que permitió ordenar sin mover los datos originales. Se utilizó una clase abstracta para la carga de datos y una interfaz para definir el método de ordenamiento (quicksort), cumpliendo con la estructura solicitada. El algoritmo QuickSort funciona seleccionando un pivote y reorganizando los elementos de tal forma que los menores quedan a un lado y los mayores al otro, repitiendo este proceso de forma recursiva.

Errores encontrados

Durante el desarrollo se presentaron algunos errores:

- Al inicio el programa no tenía interfaz ni clase abstracta.
- Se trabajó primero con ordenamiento directo en lugar de indirecto.
- Hubo errores en la lógica de la partición, lo que provocaba resultados incorrectos.
- Se utilizaban arreglos estáticos en lugar de vectores dinámicos.
- Se incluyó un menú que no era necesario para el programa.

Soluciones aplicadas

Para corregir estos errores:

- Se agregó una interfaz para definir el método quicksort.
- Se incorporó una clase abstracta para la carga de datos.
- Se corrigió la lógica de la partición para que el pivote funcionara correctamente.
- Se implementó el ordenamiento indirecto utilizando índices (idx).
- Se reemplazaron los arreglos por estructuras dinámicas (vector).
- Se eliminó el menú para simplificar la ejecución.

Resultados: El programa final permite ordenar correctamente enteros, caracteres y personas utilizando el algoritmo QuickSort. Se logró aplicar el ordenamiento indirecto sin modificar los datos originales, trabajando únicamente con índices. Además, el programa cumple con los requisitos del profesor y presenta una estructura más organizada, clara y funcional.

```

34
56
14
Cuantos caracteres: 3
e
r
t
Cuantas personas: 4
Nombre: ulises 19
Edad: Nombre: ulisa 18
Edad: Nombre: monse
Edad: 19
Nombre: carlos
Edad: 18

Enteros:
2 4 7 13 14 34 56 56

Caracteres:
e r t

Personas:
carlos - 18
ulisa - 18
ulises - 19
monse - 19

```

```

}

// ===== QUICK PERSONAS =====
int particionPersonas(int ini, int fin) {
    int pivote = personas[idxPersonas[fin]].edad;
    int i = ini - 1;

    for (int j = ini; j < fin; j++) {
        if (personas[idxPersonas[j]].edad < pivote) {
            i++;
            swap(idxPersonas[i], idxPersonas[j]);
        }
    }

    swap(idxPersonas[i + 1], idxPersonas[fin]);
    return i + 1;
}

void quickPersonas(int ini, int fin) {
    if (ini < fin) {
        int pi = particionPersonas(ini, fin);
        quickPersonas(ini, pi - 1);
        quickPersonas(pi + 1, fin);
    }
}

```

] Registro de Compilación ☒ Depuración ☐ Resultados ☒ Cerrar
 ilation results...

 rors: 0
 rnings: 0
 out: Filepath: C:\Users\Monserrat 2007\Downloads\programas\compilados\Orden

Ordenamiento de manera indirecta burbuja ordenar conjunto de caracteres, enteros y nuevo tipo de dato (PE y POO)

Introducción: En este programa trabajé con el algoritmo de ordenamiento Burbuja aplicado de forma **indirecta**, usando tres tipos de datos: enteros, caracteres y un tipo de dato definido como *Persona* (nombre y edad). La idea fue mantener el mismo enfoque de los programas anteriores, utilizando programación orientada a objetos y estructuras dinámicas. A diferencia del ordenamiento directo, aquí no se modifican los datos originales, sino que se utilizan índices para reorganizar la información.

Objetivo: Implementar el algoritmo Burbuja de forma indirecta para ordenar enteros, caracteres y personas, utilizando programación orientada a objetos y estructuras dinámicas.

Desarrollo: Primero se ingresaron los datos: números, caracteres y personas con nombre y edad. Estos datos se almacenaron en vectores para permitir un manejo dinámico. Después se crearon vectores de índices (idx) que representan las posiciones de los datos originales. El algoritmo Burbuja se aplicó sobre estos índices, intercambiándolos según el orden, en lugar de mover directamente los datos. Se utilizó una estructura basada en una clase, siguiendo el mismo estilo de los programas anteriores, para mantener consistencia en el diseño.

El flujo del programa fue:

1. Carga de datos
2. Ordenamiento con Burbuja indirecto
3. Mostrar resultados

Errores encontrados

Durante el desarrollo se presentaron los siguientes errores:

- Al inicio se realizó el ordenamiento de forma directa en lugar de indirecta.
- No se usaban correctamente los vectores de índices (idx).
- Se trabajó con arreglos estáticos en lugar de estructuras dinámicas.
- El código no seguía la misma estructura que los programas anteriores.
- Se agregaron elementos innecesarios como menús.

Soluciones aplicadas

Para corregir los errores:

- Se implementó correctamente el ordenamiento indirecto utilizando índices (idx).
- Se reemplazaron los arreglos por vector.
- Se ajustó la estructura del programa para que fuera igual a los anteriores (con POO).
- Se eliminó el menú para simplificar el flujo.
- Se organizó mejor el código para que fuera más claro y consistente.

Resultados: El programa final ordena correctamente enteros, caracteres y personas usando el algoritmo Burbuja de forma indirecta. Se logró mantener los datos originales sin modificación,

trabajando únicamente con índices. Además, el programa quedó alineado con los anteriores, tanto en funcionamiento como en estructura, cumpliendo con todos los requisitos.

```

C:\Users\Monserrat 2007\Downloads\programas corregidos\OrdenamientoEnteros POG.cpp - [Executing] - Dev-C++ 5.11
Archivo Edición Buscar Ver Proyecto Ejecutar Herramientas AStyle Ventana Ayuda
(globals)
Proyecto Clases
C:\Users\Monserrat 2007\Dov x
i
s
Cuántas personas: 5
Nombre: cristian
Edad: 19
Nombre: ara
Edad: 19
Nombre: erik
Edad: 18
Nombre: espinoza
Edad: 19
Nombre: monse
Edad: 18
Enteros ordenados:
4 54
Caracteres ordenados:
i l s u
Personas ordenadas por edad:
erik - 18
monse - 18
cristian - 19
ara - 19
espinoza - 19
Process exited after 35.28 seconds with return value 0

```

```

// ===== BURBUJA INDIRECTA =====

void ordenarEnteros() {
    for (int i = 0; i < idxEnteros.size() - 1; i++) {
        for (int j = 0; j < idxEnteros.size() - i - 1; j++) {
            if (enteros[idxEnteros[j]] > enteros[idxEnteros[j + 1]]) {
                swap(idxEnteros[j], idxEnteros[j + 1]);
            }
        }
    }
}

void ordenarCaracteres() {
    for (int i = 0; i < idxCaracteres.size() - 1; i++) {
        for (int j = 0; j < idxCaracteres.size() - i - 1; j++) {
            if (caracteres[idxCaracteres[j]] > caracteres[idxCaracteres[j + 1]]) {
                swap(idxCaracteres[j], idxCaracteres[j + 1]);
            }
        }
    }
}

```

Ordenamiento de manera indirecta merge sort ordenar conjunto de caracteres, enteros y nuevo tipo de dato (PE y POO)

Introducción: En este programa trabajé con el algoritmo de ordenamiento MergeSort aplicado de forma **indirecta**, utilizando tres tipos de datos: enteros, caracteres y un tipo de dato llamado *Persona* (nombre y edad). La idea fue seguir la misma lógica que en los programas anteriores, usando programación orientada a objetos y estructuras dinámicas. A diferencia del ordenamiento directo, aquí no se modifican los datos originales, sino que se utilizan índices para reorganizar la información, lo que hace el proceso más limpio y controlado.

Objetivo: Implementar el algoritmo MergeSort de manera indirecta para ordenar enteros, caracteres y personas, utilizando programación orientada a objetos y estructuras dinámicas.

Desarrollo: Primero se ingresaron los datos: números, caracteres y personas con nombre y edad. Estos se almacenaron en vectores para trabajar de forma dinámica. Después se crearon vectores de índices (idx), los cuales apuntan a las posiciones originales de los datos. El algoritmo MergeSort se aplicó sobre estos índices, permitiendo ordenar sin mover directamente los valores. El proceso del algoritmo se basa en dividir los datos en partes pequeñas, ordenarlas y luego unir las (merge) respetando el orden. Todo esto se hizo trabajando sobre los índices, no sobre los datos reales. Se mantuvo la misma estructura de los programas anteriores para conservar consistencia en el diseño.

El flujo fue:

1. Carga de datos
2. Aplicación de MergeSort indirecto
3. Mostrar resultados

Errores encontrados

Durante el desarrollo se presentaron varios errores:

- Se implementó inicialmente el ordenamiento de forma directa en lugar de indirecta.
- Hubo errores en la lógica del proceso de mezcla (merge).
- No se utilizaban correctamente los vectores de índices (idx).
- Se usaban arreglos estáticos en lugar de estructuras dinámicas.
- El programa no seguía la misma estructura que los anteriores.
- Se incluyó un menú innecesario.

Soluciones aplicadas

Para corregir los errores:

- Se implementó correctamente el ordenamiento indirecto usando índices (idx).
- Se corrigió la lógica del proceso de mezcla para que funcionara adecuadamente.
- Se reemplazaron los arreglos por vector.
- Se ajustó la estructura del programa para que coincidiera con los anteriores.
- Se eliminó el menú para simplificar la ejecución.
- Se organizó mejor el código para hacerlo más claro.

Resultados: El programa final permite ordenar correctamente enteros, caracteres y personas utilizando el algoritmo MergeSort de forma indirecta. Se logró mantener los datos originales sin modificarlos, trabajando únicamente con índices. Además, el programa quedó consistente con los anteriores, cumpliendo con todos los requisitos y mostrando una mejor organización en su estructura.

```

12
1
Cantidad de caracteres: 5
y
u
i
o
l
Cantidad de personas: 3
Nombre: ariel
Edad: 19
Nombre: erik
Edad: 19
Nombre: arpa
Edad: 19

Enteros ordenados:
1 8 12 56 78 90

Caracteres ordenados:
i l o u y

Personas ordenadas por edad:
ariel - 19
erik - 19
arpa - 19

-----
Process exited after 27.71 seconds with return value 0
Presione una tecla para continuar . . .

```

```

132
133
134 // ----- MERGE PERSONAS -----
135 void mergePersonas(int ini, int mid, int fin) {
136     vector<int> temp;
137     int i = ini, j = mid + 1;
138
139     while (i <= mid && j <= fin) {
140         if (personas[idxPersonas[i]].edad <= personas[idxPersonas[j]].edad)
141             temp.push_back(idxPersonas[i++]);
142         else
143             temp.push_back(idxPersonas[j++]);
144     }
145
146     while (i <= mid) temp.push_back(idxPersonas[i++]);
147     while (j <= fin) temp.push_back(idxPersonas[j++]);
148
149     for (int k = 0; k < temp.size(); k++)
150         idxPersonas[ini + k] = temp[k];
151
152 void mergeSortPersonas(int ini, int fin) {
153     if (ini < fin) {
154         int mid = (ini + fin) / 2;
155         mergeSortPersonas(ini, mid);

```

Ordenamiento de manera indirecta quick sort ordenar conjunto de caracteres, enteros y nuevo tipo de dato (PE y POO)

Introducción: En este programa trabajé con el algoritmo de ordenamiento QuickSort aplicado de forma **indirecta**, utilizando tres tipos de datos: enteros, caracteres y un tipo de dato llamado *Persona* (nombre y edad). Se siguió la misma lógica de los programas anteriores, usando programación orientada a objetos y estructuras dinámicas. La idea principal fue ordenar sin modificar los datos originales, utilizando índices que permiten reorganizar la información de forma más controlada.

Objetivo: Implementar el algoritmo QuickSort de manera indirecta para ordenar enteros, caracteres y personas, utilizando programación orientada a objetos y estructuras dinámicas.

Desarrollo: Primero se ingresaron los datos: números, caracteres y personas con nombre y edad. Estos se almacenaron en vectores para permitir un manejo dinámico. Después se crearon vectores de índices (idx) que apuntan a las posiciones originales de los datos. El algoritmo QuickSort se aplicó sobre estos índices, reorganizándolos en función de un pivote. El proceso consiste en elegir un pivote, separar los elementos menores y mayores, y repetir este proceso de forma recursiva hasta que todo quede ordenado. Todo esto se realizó trabajando sobre los índices, sin mover directamente los datos. Se mantuvo la misma estructura de los programas anteriores para conservar consistencia.

El flujo del programa fue:

1. Carga de datos
2. Aplicación de QuickSort indirecto
3. Mostrar resultados

Errores encontrados

Durante el desarrollo se presentaron algunos errores:

- Inicialmente se implementó el ordenamiento de forma directa en lugar de indirecta.
- Hubo errores en la lógica de la partición, lo que afectaba el orden final.
- No se utilizaban correctamente los vectores de índices (idx).
- Se trabajó con arreglos estáticos en lugar de estructuras dinámicas.
- El programa no seguía la misma estructura que los anteriores.
- Se incluyó un menú innecesario.

Soluciones aplicadas

Para corregir los errores:

- Se implementó correctamente el ordenamiento indirecto utilizando índices (idx).
- Se corrigió la lógica de la partición para que el pivote funcionara correctamente.
- Se reemplazaron los arreglos por vector.
- Se ajustó la estructura del programa para que fuera igual a los anteriores.
- Se eliminó el menú para simplificar la ejecución.
- Se organizó mejor el código para hacerlo más claro.

Resultados: El programa final permite ordenar correctamente enteros, caracteres y personas utilizando el algoritmo QuickSort de forma indirecta. Se logró mantener los datos originales sin modificarlos, trabajando únicamente con índices. Además, el programa cumple con todos los requisitos y mantiene una estructura clara y consistente con los demás programas.

```

Edad: 16
Nombre: ra
Edad: 19
Nombre: car
Edad: 19
Nombre: los
Edad: 19
Nombre: ju
Edad: 19

Enteros:
1 12 12 34 56 78 90

Caracteres:
e i l s s s u

Personas:
ra - 16
se - 17
ses - 18
non - 19
ili - 19
ra - 19
car - 19
los - 19
ju - 19

```

```

// ===== MÉTODO GENERAL =====
void quicksort() {
    if (!idxEnteros.empty())
        quickEnteros(0, idxEnteros.size() - 1);

    if (!idxCaracteres.empty())
        quickCaracteres(0, idxCaracteres.size() - 1);

    if (!idxPersonas.empty())
        quickPersonas(0, idxPersonas.size() - 1);
}

void mostrar() {
    cout << "\nEnteros:\n";
    for (int i = 0; i < idxEnteros.size(); i++)
        cout << enteros[idxEnteros[i]] << " ";

    cout << "\n\nCaracteres:\n";
    for (int i = 0; i < idxCaracteres.size(); i++)
        cout << caracteres[idxCaracteres[i]] << " ";

    cout << "\n\nPersonas:\n";
    for (int i = 0; i < idxPersonas.size(); i++)
        cout << personas[idxPersonas[i]].nombre
            << " - "
            << personas[idxPersonas[i]].edad << endl;
}

```